

CS6650 HW3: Threading and Load Tests

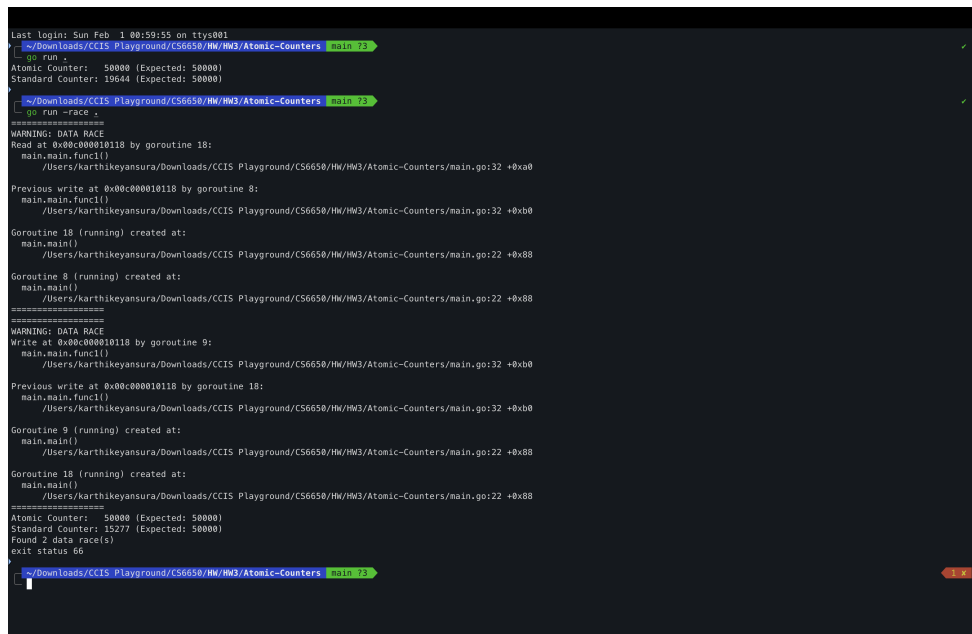
Sai Karthikeyan Sura
February 2, 2026

Part II: Thread Experiments

1. Atomicity

In this experiment, I explored race conditions inherent in shared memory access. I compared a standard integer counter against a thread-safe `atomic.Uint64` counter across 50 concurrent goroutines, each incrementing 1,000 times.

- **Values Seen:** The atomic counter consistently reached 50,000. The standard counter consistently failed, dropping to values like 19,644 (first run) and 15,277 (with race detector).
- **What is happening?** This is a **race condition**. The operation `standardOps++` is not atomic—it consists of three steps: *Read* the current value, *Modify* (add 1), and *Write* back. When goroutines interleave, they read the same value and overwrite each other's increments, causing “lost updates.”
- **The -race flag:** The Go Data Race Detector instruments the binary to monitor memory access. It identified **2 data races** at line 32, showing exactly which goroutines conflicted.



```
Last login: Sun Feb 1 00:59:55 on tty001
~/Downloads/CCIS_Playground/CS6650/HW/HW3/Atomic-Counters main 73
$ go run .
Atomic Counter: 50000 (Expected: 50000)
Standard Counter: 19644 (Expected: 50000)

~/Downloads/CCIS_Playground/CS6650/HW/HW3/Atomic-Counters main 73
$ go run -race .
WARNING: DATA RACE
Read at 0x00c000010118 by goroutine 18:
  main.main.func1()
    /Users/karthikeyansura/Downloads/CCIS_Playground/CS6650/HW/HW3/Atomic-Counters/main.go:32 +0xa0
Previous write at 0x00c000010118 by goroutine 8:
  main.main.func1()
    /Users/karthikeyansura/Downloads/CCIS_Playground/CS6650/HW/HW3/Atomic-Counters/main.go:32 +0xa0
Goroutine 18 (running) created at:
  main.main()
    /Users/karthikeyansura/Downloads/CCIS_Playground/CS6650/HW/HW3/Atomic-Counters/main.go:22 +0x88
Goroutine 8 (running) created at:
  main.main()
    /Users/karthikeyansura/Downloads/CCIS_Playground/CS6650/HW/HW3/Atomic-Counters/main.go:22 +0x88
WARNING: DATA RACE
Write at 0x00c000010118 by goroutine 9:
  main.main.func1()
    /Users/karthikeyansura/Downloads/CCIS_Playground/CS6650/HW/HW3/Atomic-Counters/main.go:32 +0xa0
Previous write at 0x00c000010118 by goroutine 18:
  main.main.func1()
    /Users/karthikeyansura/Downloads/CCIS_Playground/CS6650/HW/HW3/Atomic-Counters/main.go:32 +0xa0
Goroutine 9 (running) created at:
  main.main()
    /Users/karthikeyansura/Downloads/CCIS_Playground/CS6650/HW/HW3/Atomic-Counters/main.go:22 +0x88
Goroutine 18 (running) created at:
  main.main()
    /Users/karthikeyansura/Downloads/CCIS_Playground/CS6650/HW/HW3/Atomic-Counters/main.go:22 +0x88
Atomic Counter: 50000 (Expected: 50000)
Standard Counter: 15277 (Expected: 50000)
Found 2 data race(s)
exit status 66
```

Figure 1: Standard counter lost 61–69% of updates; race detector identified conflicts.

2. Collections (Plain Map Crash)

I attempted to write to a standard Go `map[int]int` concurrently from 50 goroutines.

- **Result:** The program crashed with fatal error: `concurrent map writes`.
- **Why?** Go maps are not designed for concurrent writes. Unlike the counter (which silently gives wrong results), Go's runtime **actively detects** concurrent map access and terminates the program. This is a safety feature—it's better to crash than silently corrupt the map's internal hash table structure (buckets, pointers, metadata).

```
Last login: Sun Feb 1 01:53:53 on ttys001
~/Downloads/CCIS_Playground/CS6650/HM/HM3/collections main 73
$ go run -x
Starting concurrent map writes...
fatal error: concurrent map writes

goroutine 26 [running]:
internal/runtime/maps.fatal({0x1022893e67, 0x0?})
/opt/homebrew/Cellar/go/1.25.5/libexec/src/runtime/panic.go:1046 +0x20
main.main.func1(0x0)
/Users/karthikeyansura/Downloads/CCIS_Playground/CS6650/HM/HM3/collections/main.go:27 +0x68
created by main.main in goroutine 1
/Users/karthikeyansura/Downloads/CCIS_Playground/CS6650/HM/HM3/collections/main.go:19 +0x80

goroutine 1 [runnable]:
sync.(*WaitGroup).Add(0x140000a0010, 0x1)
/opt/homebrew/Cellar/go/1.25.5/libexec/src/sync/waitgroup.go:77 +0x274
main.main()
/Users/karthikeyansura/Downloads/CCIS_Playground/CS6650/HM/HM3/collections/main.go:17 +0xa0

goroutine 18 [runnable]:
main.main.gowrap1()
/Users/karthikeyansura/Downloads/CCIS_Playground/CS6650/HM/HM3/collections/main.go:19
runtime.goexit({})
/opt/homebrew/Cellar/go/1.25.5/libexec/src/runtime/asm_arm64.s:1268 +0x4
created by main.main in goroutine 1
/Users/karthikeyansura/Downloads/CCIS_Playground/CS6650/HM/HM3/collections/main.go:19 +0x80

goroutine 19 [runnable]:
main.main.gowrap1()
/Users/karthikeyansura/Downloads/CCIS_Playground/CS6650/HM/HM3/collections/main.go:19
runtime.goexit({})
/opt/homebrew/Cellar/go/1.25.5/libexec/src/runtime/asm_arm64.s:1268 +0x4
created by main.main in goroutine 1
/Users/karthikeyansura/Downloads/CCIS_Playground/CS6650/HM/HM3/collections/main.go:19 +0x80

goroutine 20 [runnable]:
internal/runtime/maps.(*Map).putSlotSmallFast64(0x14000020007, 0x102248a007, 0x8ace0d0000b1bfd7, 0x7d07)
/opt/homebrew/Cellar/go/1.25.5/libexec/src/internal/runtime/maps/runtime_fast64_swiss.go:154 +0x170
main.main.func1(0x2)
/Users/karthikeyansura/Downloads/CCIS_Playground/CS6650/HM/HM3/collections/main.go:27 +0x68
created by main.main in goroutine 1
/Users/karthikeyansura/Downloads/CCIS_Playground/CS6650/HM/HM3/collections/main.go:19 +0x80

goroutine 21 [runnable]:
main.main.gowrap1()
/Users/karthikeyansura/Downloads/CCIS_Playground/CS6650/HM/HM3/collections/main.go:19
runtime.goexit({})
/opt/homebrew/Cellar/go/1.25.5/libexec/src/runtime/asm_arm64.s:1268 +0x4
created by main.main in goroutine 1
/Users/karthikeyansura/Downloads/CCIS_Playground/CS6650/HM/HM3/collections/main.go:19 +0x80

goroutine 22 [runnable]:
main.main.gowrap1()
/Users/karthikeyansura/Downloads/CCIS_Playground/CS6650/HM/HM3/collections/main.go:19
```

Figure 2: Crash caused by concurrent map writes.

3. Thread-Safe Collections: Mutex, RWMutex, Sync.Map

I compared three approaches to fixing the map crash, running each 3 times.

Approach	Iteration 1	Iteration 2	Iteration 3	Mean	Map Size
Mutex	15.81ms	14.62ms	8.58ms	13.00ms	50,000
RWMutex	13.65ms	12.52ms	7.91ms	11.36ms	50,000
sync.Map	5.39ms	3.84ms	2.36ms	3.86ms	50,000

Table 1: Thread-safe collection performance comparison.

Results & Tradeoffs:

1. **Mutex ($\approx 13\text{ms}$):** Uses a global exclusive lock. Simple but serializes *all* operations, becoming a bottleneck under high contention.
2. **RWMutex ($\approx 11\text{ms}$):** Similar to Mutex in this *write-only* experiment because `Lock()` (write lock) is exclusive. RWMutex excels when **reads dominate**—multiple readers can hold `RLock()` simultaneously.
3. **sync.Map ($\approx 3.9\text{ms}$):** Significantly faster ($\sim 3.4\text{x}$ speedup). Optimized for cases where keys are stable or writes are disjoint. Since each goroutine wrote to unique keys ($g*1000+i$), sync.Map’s internal sharding handled this efficiently.

What if reads dominate? RWMutex would significantly outperform Mutex because concurrent readers don’t block each other. This is directly applicable to the Locust experiment with a 3:1 GET:POST ratio.

```
Last login: Sun Feb 1 02:34:44 on tty001
~/Downloads/CCIS_Playground/CS6658/HW/HW3/Mutex main +3 14 72
go run . 66 go run . 66 go run .
Map size [Mutex]: 50000 (Expected: 50000)
Time: 15.88775ms
Map size [Mutex]: 50000 (Expected: 50000)
Time: 14.62425ms
Map size [Mutex]: 50000 (Expected: 50000)
Time: 8.57587ms
^
~/Downloads/CCIS_Playground/CS6658/HW/HW3/RWMutex main +3 14 72
cd ../RWMutex 66 go run . 66 go run . 66 go run .
Map size [RWMutex]: 50000 (Expected: 50000)
Time: 13.646042ms
Map size [RWMutex]: 50000 (Expected: 50000)
Time: 12.51867ms
Map size [RWMutex]: 50000 (Expected: 50000)
Time: 7.90675ms
^
~/Downloads/CCIS_Playground/CS6658/HW/HW3/RWMutex main +3 14 72
cd ../Sync.Map 66 go run . 66 go run . 66 go run .
Map size [Sync.Map]: 50000 (Expected: 50000)
Time: 5.38578ms
Map size [Sync.Map]: 50000 (Expected: 50000)
Time: 3.835ms
Map size [Sync.Map]: 50000 (Expected: 50000)
Time: 2.357542ms
~/Downloads/CCIS_Playground/CS6658/HW/HW3/Sync.Map main +3 14 72
```

Figure 3: Performance comparison: sync.Map is 3.4x faster for disjoint key writes.

4. File Access

I measured the time to write 100,000 lines (52 bytes each, 4.96 MB total) with and without buffering.

Mode	Iteration 1	Iteration 2	Iteration 3	Mean
Unbuffered	113.49ms	86.72ms	86.42ms	95.54ms
Buffered	3.44ms	3.28ms	3.42ms	3.38ms

Table 2: File I/O performance: unbuffered vs buffered.

Speedup: ~28x faster with buffering!

Lesson: System calls are expensive. Each `f.Write()` triggers a syscall, switching from User Mode → Kernel Mode → User Mode. Unbuffered writes make ~100,000 syscalls; buffered writes accumulate data in a 4KB memory buffer and write in large chunks, reducing syscalls to ~1,200. This principle underlies **write-ahead logging** in databases and **batched commits** in distributed systems. The tradeoff: buffered writes may lose data on crashes before flush.

```
Last login: Sun Feb 1 03:54:26 on ttys002
~/Downloads/CCIS_Playground/CS6650/HW/HW3/File-Access main #2 12 72 ✓
go run -c
Writing 100000 lines (52 bytes each)
Total data: 4.96 MB

Unbuffered writes:
Iteration 1: 113.492708ms
Iteration 2: 86.722541ms
Iteration 3: 86.423953ms

Buffered writes:
Iteration 1: 3.444584ms
Iteration 2: 3.280083ms
Iteration 3: 3.422167ms
```

Figure 4: 28x speedup with buffered I/O due to reduced syscalls.

5. Context Switching

I measured the cost of goroutine context switches via 1,000,000 round trips (2,000,000 switches) over unbuffered channels.

Mode	Avg Total Time	Avg Switch Time	Relative
Single-threaded (GOMAXPROCS=1)	~136ms	~ 68ns	1.0x
Multi-threaded (12 cores)	~204ms	~ 102ns	1.5x slower

Table 3: Goroutine context switch times.

Why is single-threaded faster? Paradoxically, restricting to one OS thread is faster for this synchronous round trips task:

- **Single thread:** Pure user-space stack pointer swap; no OS involvement.
- **Multi-thread:** CPU cache coherence across cores, potential thread migration, memory barriers.

Context Switching Cost Hierarchy: Goroutines ($\sim 70\text{ns}$) $\ll$ OS Threads ($\sim 1\text{--}10\mu\text{s}$) $\ll$ Processes ($\sim 10\text{--}100\mu\text{s}$) $\ll$ VMs ($\sim 100\mu\text{s}\text{--}1\text{ms}$). This explains why Go can handle millions of concurrent connections, and why Locust uses green threads for efficient load generation.

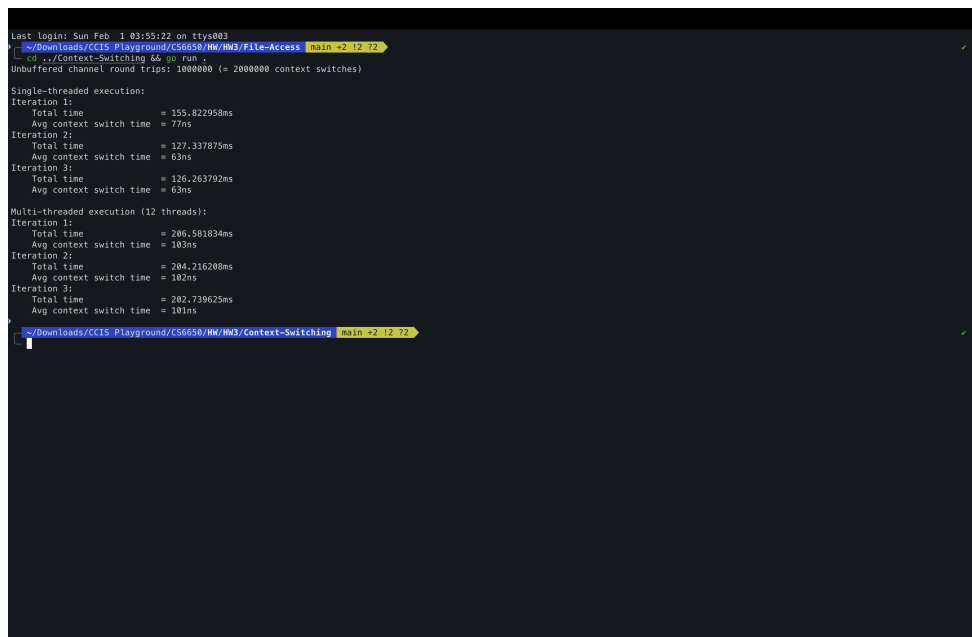


Figure 5: Single-thread goroutine switching is 1.5x faster than multi-thread.

Part III: Making Threads work hard with Load-Testing

Server Setup: Applying Part II

The test server is a Go Gin API (based on HW1b) modified to be **thread-safe** using `sync.RWMutex`—directly applying the lessons from the Collections experiment:

```

1 type AlbumStore struct {
2     sync.RWMutex
3     albums []album
4 }
5
6 func getAlbums(c *gin.Context) {
7     store.RLock()           // Read lock - concurrent readers allowed
8     defer store.RUnlock()
9     c.IndentedJSON(http.StatusOK, store.albums)
10 }
11
12 func postAlbums(c *gin.Context) {
13     store.Lock()           // Write lock - exclusive access
14     store.albums = append(store.albums, newAlbum)
15     store.Unlock()
16 }

```

Listing 1: Thread-safe AlbumStore using RWMutex

RWMutex is ideal for the 3:1 GET:POST workload since GET operations can proceed concurrently.

1. Baseline Test (1 Worker, 50 Users)

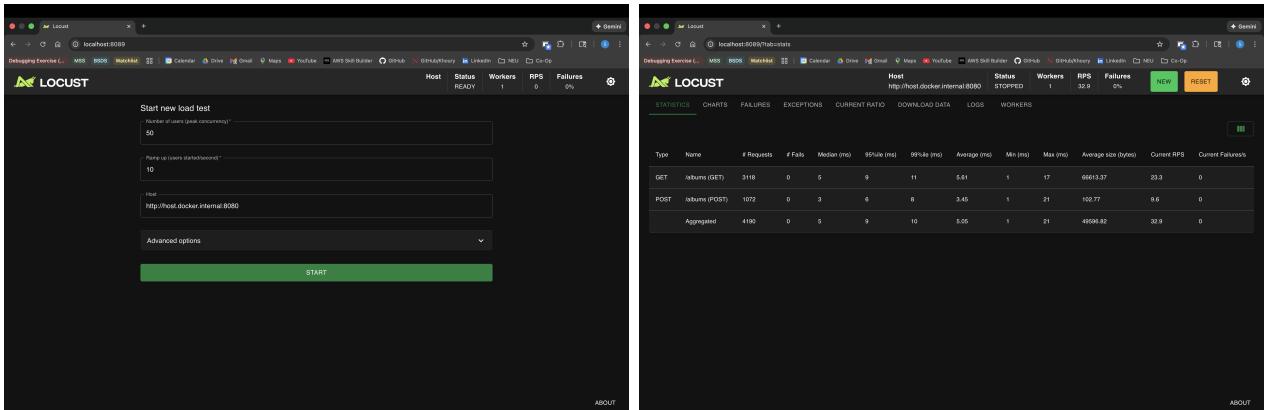


Figure 6: Baseline load test with 1 worker.

Type	# Requests	Median	95%ile	Avg	Avg Size	RPS
GET /albums	3,118	5ms	9ms	5.61ms	66.6 KB	23.3
POST /albums	1,072	5ms	6ms	3.45ms	103 B	9.6
Aggregated	4,190	5ms	9ms	5.05ms	49.6 KB	32.9

Table 4: Baseline test statistics (1 worker, 50 users).

Analysis:

- **Request Ratio:** GET:POST $\approx$ 3:1 as configured (3,118 : 1,072).
- **Zero Failures:** The RWMutex-protected server handles concurrent load correctly.
- **GET Response Size Growing:** Each POST adds an album, so GET returns an increasingly large JSON list (66.6 KB average).

2. Amdahl's Law Test (4 Workers, 200 Users)

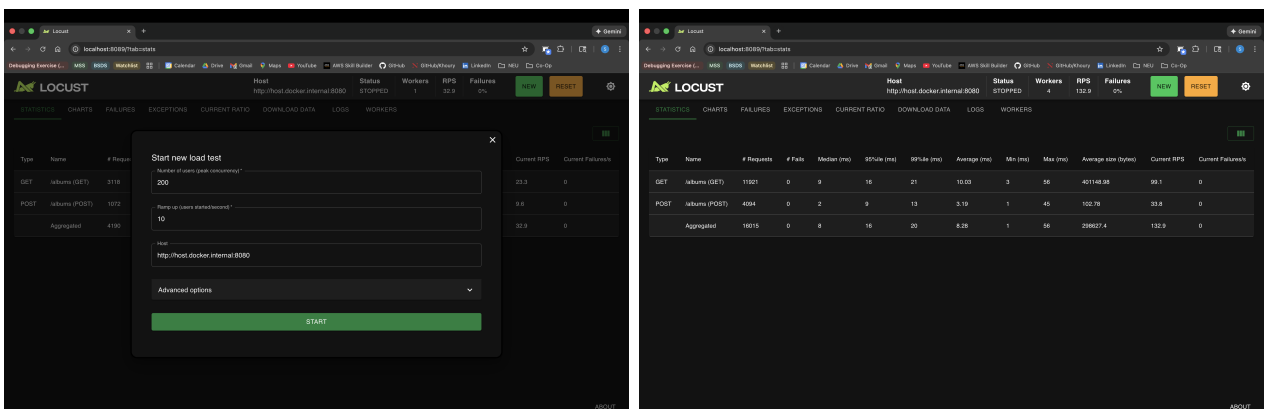


Figure 7: Baseline load test with 1 worker.

Metric	1 Worker (50 users)	4 Workers (200 users)	Scaling
Total RPS	32.9	132.9	4.04x
GET RPS	23.3	99.1	4.25x
POST RPS	9.6	33.8	3.52x

Table 5: Worker scaling analysis.

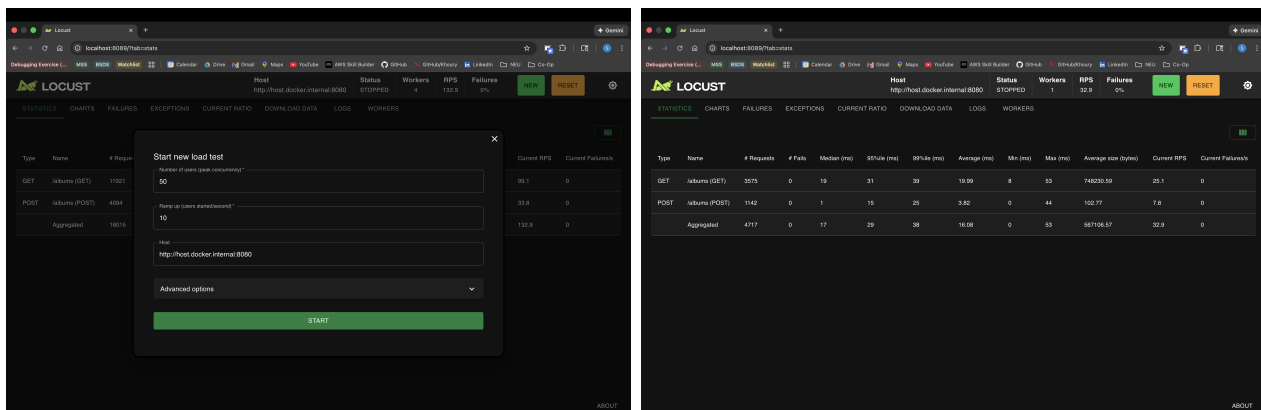
Analysis: The scaling was **nearly linear** ($33 \times 4 \approx 132$). This proves:

- The **Go server was NOT the bottleneck**—it handled quadrupled load without degradation.
- The bottleneck was the **Locust client** itself (Python’s GIL limits single-process request generation).
- Adding workers removed the client-side bottleneck.

Where would Amdahl’s Law limit scaling? If POST operations dominated, the write lock would serialize incoming requests, significantly reducing concurrency. Performance would also degrade if the server had a single-threaded bottleneck. Additionally, overall throughput could be limited by network bandwidth saturation, preventing the system from handling higher request rates even if other resources were available.

3. FastHttpUser Test

I switched from `HttpUser` (Python `requests` library) to `FastHttpUser` (C-based `geventhttpclient`) to test if HTTP client overhead was limiting performance.



(a) FastHttpUser configuration

(b) Results: 32.9 RPS, unchanged

Figure 8: FastHttpUser test results.

Type	# Requests	Median	Avg	Avg Size
GET /albums	3,575	19ms	19.99ms	748 KB
POST /albums	1,142	15ms	3.82ms	103 B
Aggregated	4,717	17ms	16.08ms	567 KB

Table 6: FastHttpUser statistics

Analysis:

- **Throughput unchanged:** ~ 32.9 RPS, same as baseline.
- **GET latency higher:** 19ms vs 5ms in baseline—because this test ran *after* the Amdahl's Law test, the album list had grown to $\sim 5,000$ entries (748 KB response).
- **Why no improvement?** The test was limited by the configured `wait_time = between(1, 2)` seconds, not by HTTP client CPU overhead. Optimization only helps when applied to the *actual bottleneck*.

According to Locust documentation, FastHttpUser can achieve 5–6x higher throughput when the bottleneck is client-side CPU, enabling $\sim 16,000$ RPS vs $\sim 4,000$ RPS with HttpUser on the same hardware.

4. Locust Charts



Figure 9: Locust charts showing: (1) RPS spike to ~ 130 with 4 workers, (2) Response times increasing as album list grows, (3) User count across test runs.

GIN	2026/02/01 - 10:59:35	200	2.68333ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:35	200	2.582791ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:35	200	1.653125ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:35	200	2.274375ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:36	200	1.163389ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:36	200	1.417291ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:36	200	1.293375ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:36	201	53.633μs	127.0.0.1	POST	"/albums"
GIN	2026/02/01 - 10:59:36	200	2.837334ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:36	201	73.75μs	127.0.0.1	POST	"/albums"
GIN	2026/02/01 - 10:59:36	201	1.243292ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:36	201	92.333μs	127.0.0.1	POST	"/albums"
GIN	2026/02/01 - 10:59:36	200	2.389541ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:36	200	97.541μs	127.0.0.1	POST	"/albums"
GIN	2026/02/01 - 10:59:36	200	1.25975ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:36	200	1.452842ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:36	200	2.838534ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:36	200	1.419312ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:36	200	1.374291ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:36	201	97.541μs	127.0.0.1	POST	"/albums"
GIN	2026/02/01 - 10:59:36	200	1.314292ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:36	200	1.566417ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:36	201	123.209μs	127.0.0.1	POST	"/albums"
GIN	2026/02/01 - 10:59:36	200	1.61775ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:36	200	1.528842ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:36	201	76.417μs	127.0.0.1	POST	"/albums"
GIN	2026/02/01 - 10:59:36	200	3.824288ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:36	200	2.293583ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:36	200	1.364917ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:36	200	1.486789ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:36	200	2.825417ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:36	200	1.688875ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:36	201	91.233μs	127.0.0.1	POST	"/albums"
GIN	2026/02/01 - 10:59:36	201	375.166μs	127.0.0.1	POST	"/albums"
GIN	2026/02/01 - 10:59:36	200	1.958417ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:36	200	2.21ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:36	200	1.757917ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:36	200	2.689883ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:36	200	2.337542ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:36	200	3.78875ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:36	201	94.25μs	127.0.0.1	POST	"/albums"
GIN	2026/02/01 - 10:59:37	200	1.778916ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:37	201	54.75μs	127.0.0.1	POST	"/albums"
GIN	2026/02/01 - 10:59:37	200	1.559791ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:37	200	2.711288ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:37	201	79.288μs	127.0.0.1	POST	"/albums"
GIN	2026/02/01 - 10:59:37	200	1.813288ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:37	200	1.949125ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:37	200	3.3525ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:37	200	2.618867ms	127.0.0.1	GET	"/albums"
GIN	2026/02/01 - 10:59:37	201	111.667μs	127.0.0.1	POST	"/albums"
GIN	2026/02/01 - 10:59:37	201	67.667μs	127.0.0.1	POST	"/albums"

Figure 10: Gin server logs: POST (201) completes in 50–375μs; GET (200) takes 1–3ms due to JSON serialization.

Server-side Analysis: The logs reveal that at the server level, **POST is actually faster** than GET:

- **POST:** 50–375μs (simple append operation with write lock)
- **GET:** 1–3ms (must serialize entire album list to JSON)

The Locust-reported latency includes network round-trip and JSON payload transfer. As the album list grows, GET response size increases, explaining the rising response times in the charts.